# LAPORAN PRAKTIKUM
# PEMROGRAMAN BERORIENTASI OBJEK
# MODUL 7 – ABSTRACT DAN INTERFACE

NAMA : Elok Elysia Dewi

NIM : 103132400013

KELAS : S1SD-05-01

**PROGRAM STUDI SAINS DATA**

**FAKULTAS INFORMATIKA**

**TELKOM UNIVERSITY PURWOKERTO**

**2026**

A. Dasar Teori

## 1. Kelas Abstract

Kelas abstract adalah kelas yang dideklarasikan dengan kata kunci abstract dan digunakan sebagai kerangka umum bagi kelas-kelas turunannya. Kelas ini dapat memuat *abstract method*, yaitu method yang hanya dideklarasikan tanpa implementasi (tanpa body). Karena sifatnya yang belum lengkap, kelas abstract tidak dapat diinstansiasi menjadi object secara langsung. Kelas abstract hanya dapat dimanfaatkan melalui kelas turunan (*subclass*) yang meng-*override* seluruh abstract method-nya, dan kelas turunan inilah yang dapat dibentuk menjadi object.

Sebagai contoh, kelas Employee merepresentasikan konsep umum seorang pegawai dan memiliki abstract method earnings(). Kelas SalariedEmployee, CommissionEmployee, dan HourlyEmployee mewarisi Employee dan masing-masing mengimplementasikan earnings() sesuai cara penghitungan pendapatannya. Dalam diagram UML, nama kelas abstract ditulis dengan huruf miring.

Kelas abstract tetap dapat memiliki atribut, constructor, dan method biasa yang sudah berisi implementasi, sehingga tampak seperti kelas pada umumnya.

## 2. Interface

Interface mirip dengan kelas, tetapi berfungsi sebagai kontrak yang mendefinisikan method atau konstanta yang harus diimplementasikan oleh kelas lain. Jika kelas mendefinisikan kemampuan sebuah object, interface mendefinisikan seperangkat karakteristik yang harus dimiliki object tersebut. Misalnya radio, TV, dan speaker komputer sama-sama memiliki pengontrol volume, sehingga ketiganya dapat mengimplementasikan interface VolumeControl.

Hubungan antara kelas dan interface disebut realization, yaitu ketika sebuah kelas merealisasikan (mengimplementasikan) method yang dideklarasikan pada interface menggunakan kata kunci implements. Contohnya, kelas Invoice mengimplementasikan interface Payable dengan menyediakan isi dari method getPaymentAmount().

Pada Java 8, interface memiliki beberapa ketentuan:

1. Semua atribut bersifat public, static, dan final (berperan sebagai konstanta).
2. Selain abstract method, interface dapat memiliki default method dan static method.
3. Interface tidak boleh memiliki constructor.

Berbeda dengan kelas biasa, interface mendukung multiple inheritance, yaitu satu kelas dapat mengimplementasikan lebih dari satu interface sekaligus, sedangkan sebuah kelas hanya boleh memiliki satu kelas induk.

## 3. Default Method dan Static Method pada Interface

Sebelum Java 8, interface hanya dapat berisi abstract method yang secara default bersifat public dan abstract. Sejak Java 8, interface dapat memiliki:

- **Default method**, yaitu method dengan body yang ditandai kata kunci default. Method ini memungkinkan pengembang menambahkan method baru ke interface

tanpa memaksa kelas-kelas yang sudah mengimplementasikan interface tersebut untuk mengubah kodenya.

- **Static method**, yaitu method yang dimiliki oleh interface itu sendiri dan dipanggil melalui nama interface (contoh: MyInterface.anotherNewMethod()). Method ini tidak diwariskan dan tidak dapat di-*override* oleh kelas implementasi.

**4. Perbedaan Abstract Class dan Interface**

Abstract class dan interface sama-sama digunakan untuk membuat kerangka yang harus dilengkapi oleh kelas lain, tetapi keduanya memiliki beberapa perbedaan mendasar. Pertama, abstract class dapat berisi method abstract maupun method yang sudah memiliki implementasi, sedangkan method pada interface pada dasarnya tidak memiliki body, kecuali default method dan static method yang mulai diperkenalkan pada Java 8.

Kedua, abstract class dapat mendeklarasikan variabel seperti kelas biasa, sedangkan interface hanya dapat mendefinisikan konstanta yang bersifat public static final. Ketiga, abstract class merupakan bagian dari hierarki pewarisan kelas sehingga dapat diturunkan dari abstract class induknya, sedangkan interface tidak memiliki hubungan inheritance langsung dengan kelas tertentu.

Keempat, abstract class tidak mendukung pewarisan jamak (*multiple inheritance*), sedangkan interface memungkinkan sebuah kelas mengimplementasikan lebih dari satu interface sekaligus. Terakhir, abstract class boleh memiliki constructor dan diturunkan menggunakan kata kunci extends, sedangkan interface tidak boleh memiliki constructor dan diimplementasikan menggunakan kata kunci implements.

B. Guided

1. Employee

```
package Guided.Guided1;


public abstract class Employee {
    private String name;


    // three-argument constructor
    public Employee( String name )
    {
        this.name = name;
    }
```

```
    public String getName()
    {
        return name;
    }

    // Setiap pegawai pasti punya gaji, tapi rumusnya beda-beda
    public abstract double earnings();
}
```

SalariedEmployee

```
package Guided.Guided1;

public class SalariedEmployee extends Employee {
    private double weeklySalary;

    public SalariedEmployee(String name, double weeklySalary) {
        super(name); // Memanggil constructor milik superclass Employee
        this.weeklySalary = weeklySalary;
    }

    @Override
    public double earnings() {
        return weeklySalary;
    }
}
```

Main

```
package Guided.Guided1;


public class Main {

    public static void main(String[] args) {

        // Polimorfisme: referensi abstract class Employee, objek SalariedEmployee

        SalariedEmployee salaried = new SalariedEmployee("Elysia Reksy",
5000000.00);



        System.out.println("Nama: " + salaried.getName());

        System.out.println("Gaji: Rp " + salaried.earnings());

    }

}
```

Output :

```
run:
Nama: Elysia Reksy
Gaji: Rp 5000000.0
BUILD SUCCESSFUL (total time: 0 seconds)
```

2. Employee

```
package Guided.Guided2;

public abstract class Employee implements Payable {
    private String name;

    // three-argument constructor
    public Employee( String name )
    {
        this.name = name;
    }

    public String getName()
    {
        return name;
```

```
    }

    // Setiap pegawai pasti punya gaji, tapi rumusnya beda-beda
    public abstract double earnings();

    // Saat ada penagihan, silahkan langsung ke metode kalkulasi gaji (earnings)
    @Override
    public double getPaymentAmount() {
        return earnings();
    }
}
```

Invoice

```
package Guided.Guided2;

// Benda mati non-pegawai, tapi tetap diikat oleh aturan
public class Invoice implements Payable {
    private String partName;
    private int quantity;
    private double pricePerItem;

    // three-argument constructor
    public Invoice(String partName, int quantity, double pricePerItem) {
        this.partName = partName;
        this.quantity = quantity;
        this.pricePerItem = pricePerItem;
    }

    public String getPartName() {
        return partName;
    }

    // Mengimplementasikan kontrak method dari interface Payable
    //Hitung tagihan
    @Override
    public double getPaymentAmount() {
        return quantity * pricePerItem;
    }
}
```

Payable

```
package Guided.Guided2;

public interface Payable {
```

```
    double getPaymentAmount(); // method abstract tanpa body
}
```

SalariedEmployee

```
package Guided.Guided2;

public class SalariedEmployee extends Employee {
    private double weeklySalary;

    public SalariedEmployee(String name, double weeklySalary) {
        super(name); // Memanggil constructor milik superclass Employee
        this.weeklySalary = weeklySalary;
    }

    @Override
    public double earnings() {
        return weeklySalary;
    }
}
```

Main

```
package Guided.Guided2;

public class Main {
    public static void main(String[] args) {
        // Polimorfisme: Array bertipe interface Payable
        Payable[] items = new Payable[2];

        // 1. Objek Invoice (partName, quantity, pricePerItem)
        items[0] = new Invoice("Monitor LED", 2, 1500000);

        // 2. Objek SalariedEmployee (name, weeklySalary)
        items[1] = new SalariedEmployee("Raymundus Ariel Abas", 5000000);

        for (Payable item : items) {
                    // Pemanggilan polimorfis method getPaymentAmount() dari
interface Payable
            System.out.println("Jumlah yang perlu dibayar: Rp" +
item.getPaymentAmount());
        }
    }
}
```

Output :

```
run:
Jumlah yang perlu dibayar: Rp3000000.0
Jumlah yang perlu dibayar: Rp5000000.0
BUILD SUCCESSFUL (total time: 0 seconds)
```

3. Employee

```
package Guided.Guided3;

public abstract class Employee implements Payable {
    private String name;

    public Employee(String name) {
        this.name = name;
    }

    public String getName() {
        return name;
    }

    public abstract double earnings();

    @Override
    public double getPaymentAmount() {
        return earnings();
    }
}
```

Invoice

```
package Guided.Guided3;

public class Invoice implements Payable {
    private String partName;
    private int quantity;
    private double pricePerItem;

    public Invoice(String partName, int quantity, double pricePerItem) {
        this.partName = partName;
        this.quantity = quantity;
        this.pricePerItem = pricePerItem;
    }
```

```
    public String getPartName() {
        return partName;
    }

    @Override
    public double getPaymentAmount() {
        return quantity * pricePerItem;
    }
}
```

Payable

```
package Guided.Guided3;

public interface Payable {
    double getPaymentAmount();

    default void printReceipt() {
        System.out.println("Struk Resmi - Total: Rp" + getPaymentAmount());
    }

    static void logTransaction(String message) {
        System.out.println("[AUDIT LOG] " + message);
    }
}
```

SalariedEmployee

```
package Guided.Guided3;

public class SalariedEmployee extends Employee {
    private double weeklySalary;

    public SalariedEmployee(String name, double weeklySalary) {
        super(name);
        this.weeklySalary = weeklySalary;
    }

    @Override
    public double earnings() {
        return weeklySalary;
    }
}
```

Main

```
package Guided.Guided3;

public class Main {
    public static void main(String[] args) {
        // 1. Memanggil STATIC METHOD pertama
        Payable.logTransaction("Sistem Pembayaran diinisialisasi");

        Payable[] items = new Payable[2];
        items[0] = new Invoice("Ram DDR 5 16GB", 2, 2000000);
        items[1] = new SalariedEmployee("Raymundus Ariel Abas", 5000000);

        // 2. Looping hanya untuk mencetak struk masing-masing objek
        for (Payable item : items) {
            item.printReceipt();
        }

        // 3. Memanggil STATIC METHOD di luar loop agar hanya muncul sekali di
akhir
        Payable.logTransaction("Tagihan selesai diproses.");
    }
}
```

Output:

```
run:
[AUDIT LOG] Sistem Pembayaran diinisialisasi
Struk Resmi - Total: Rp4000000.0
Struk Resmi - Total: Rp5000000.0
[AUDIT LOG] Tagihan selesai diproses.
BUILD SUCCESSFUL (total time: 0 seconds)
```

C. Unguided

Soal

Sebuah bioskop memerlukan sistem kasir yang mampu memproses pembayaran dalam satu keranjang belanja. Di kasir, penonton tidak hanya membeli **Tiket Nonton**, tetapi juga bisa membeli makanan/minuman (**Paket Snack F&B**).

Secara konsep sistem, **Tiket Bioskop** dan **Paket Snack** berasal dari silsilah keluarga (hierarki) yang sepenuhnya berbeda. Tiket film murni pun bersifat abstrak (tidak bisa dibeli tanpa jenis studio). Namun, di mata program kasir, tiket dan makanan harus bisa disatukan, dihitung total tagihannya, dan dicetak di dalam satu struk secara otomatis.

Pengerjaan soal ini dibagi menjadi 4 tugas utama untuk melatih abstraksi kelas, antarmuka (*interface*), dan fitur modern Java 8+.

**Tugas 1: Fondasi Kelas Abstrak (*Abstract Class*)**

Buatlah hierarki kelas tiket bioskop dengan spesifikasi berikut:

1. Kelas Induk TiketBioskop (Wajib abstract)
    - Atribut (Private): judulFilm (String), hargaDasar (double), jumlahKursi (int).
    - Konstruktor: Menerima dan menginisialisasi ketiga atribut tersebut.
    - Method: Buat *getter* untuk ketiga atribut.
    - Abstract Method:
        - public abstract double hitungTotalBayar();
        - public abstract String getNamaTransaksi(); *(Beban perumusan kedua metode ini diserahkan kepada kelas turunannya agar output di struk nanti spesifik untuk tiap jenis tiket)*.
2. Kelas Turunan TiketReguler
    - Atribut Tambahan (Private): nomorStudio (int).
    - Konstruktor: Menginisialisasi atribut induk via super(judulFilm, hargaDasar, jumlahKursi) dan atribut lokal nomorStudio.
    - Override Method:
        - hitungTotalBayar(): Mengembalikan hasil hargaDasar * jumlahKursi.
        - getNamaTransaksi(): Mengembalikan string "Tiket Reguler (Studio " + nomorStudio + ") - " + getJudulFilm() + " (" + getJumlahKursi() + " x Rp" + getHargaDasar() + ")".

3. Kelas Turunan TiketVIP
    - Atribut Tambahan (Private): biayaFasilitasVIP (double). *(Catatan: Biaya fasilitas seperti sofa recliner ini dikenakan untuk setiap kursi)*.
    - Konstruktor: Menginisialisasi atribut induk via super(...) dan atribut lokal biayaFasilitasVIP.
    - Override Method:
        - hitungTotalBayar(): Mengembalikan hasil (hargaDasar + biayaFasilitasVIP) * jumlahKursi.
        - getNamaTransaksi(): Mengembalikan string "Tiket VIP Premiere - " + getJudulFilm() + " (" + getJumlahKursi() + " x Rp" + (getHargaDasar() + biayaFasilitasVIP) + ")"

**Tugas 2: Kontrak Antarmuka Lintas Entitas (*Interface*)**

Sistem kasir membutuhkan standar publik agar setiap barang bisa diproses.

1. Antarmuka ItemTransaksi Buatlah *interface* bernama ItemTransaksi dengan dua deklarasi kontrak (metode abstrak):
    - double getNominalBayar();
    - String getNamaTransaksi();
2. Integrasi Kelas TiketBioskop ke Antarmuka
    - Tambahkan implements ItemTransaksi pada deklarasi kelas TiketBioskop.
    - Penting (Alasan Arsitektural): Metode hitungTotalBayar() adalah logika internal spesifik tiket, sedangkan getNominalBayar() adalah kontrak publik kasir. Wujudkan metode getNominalBayar() di dalam TiketBioskop yang isinya sekadar me-*return* nilai dari hitungTotalBayar().
    - *(Catatan: Anda tidak perlu menata ulang getNamaTransaksi() di sini karena ia sudah dideklarasikan sebagai metode abstrak pada Tugas 1)*.
3. Kelas Baru: PaketSnack (Benda Non-Tiket) Buat kelas independen bernama PaketSnack yang bukan turunan TiketBioskop, tetapi mengimplementasikan ItemTransaksi.
    - Atribut (Private): namaPaket (String), jumlahPorsi (int), hargaPerPorsi (double).
    - Konstruktor: Menginisialisasi ketiga atribut.
    - Override Method:
        - getNominalBayar(): Mengembalikan hasil jumlahPorsi * hargaPerPorsi.
        - getNamaTransaksi(): Mengembalikan string "F&B Snack: " + namaPaket + " (" + jumlahPorsi + " x Rp" + hargaPerPorsi + ")".

**Tugas 3: Modernisasi Antarmuka Java 8+**

Suntikkan fitur tambahan ke antarmuka ItemTransaksi tanpa mengharuskan kelas-kelas di atas dimodifikasi ulang:

1. Default Method Tambahkan metode default void cetakRincianItem() yang bertugas mencetak format baris struk ke layar. Panggil metode milik antarmuka (getNamaTransaksi dan getNominalBayar) untuk mengambil data secara dinamis. Format wajib: - [Nama Transaksi (Kuantitas x Harga Satuan)] | Total: Rp[Nominal Bayar]
2. Static Method Tambahkan metode static void catatLogKasir(String pesan) yang mencetak teks: [LOG SISTEM KASIR] [pesan]

**Tugas 4: Pengujian Polimorfisme di Main**

Tuliskan alur berikut di dalam *method* main:

1. Uji Proteksi Abstraksi: Tuliskan baris kode instansiasi TiketBioskop (misal: new TiketBioskop(...)), lalu jadikan baris tersebut sebagai komentar (//).
2. Buka Kasir: Panggil metode statis catatLogKasir("Membuka transaksi baru untuk Pelanggan #042."); langsung melalui nama antarmukanya.
3. Buat Objek Transaksi:
    - 1 objek TiketReguler (Film "Interstellar", harga dasar 40000, 2 kursi, Studio 3).
    - 1 objek TiketVIP (Film "Dune: Part Two", harga dasar 60000, 2 kursi, biaya VIP 25000).
    - 1 objek PaketSnack ("Combo Popcorn Caramel + 2 Soda", 1 porsi, harga 55000).
4. Array Polimorfik: Simpan ketiga objek tersebut ke dalam satu array heterogen bertipe ItemTransaksi[].
5. Pemrosesan Dinamis (Loop):
    - Gunakan perulangan (*for* atau *for-each*) terhadap array tersebut.
    - Di dalam perulangan, panggil *default method* cetakRincianItem() lewat masing-masing objek/elemen array.
    - Hitung dan jumlahkan nominal bayar keseluruhan belanjaan secara akumulatif.
6. Tutup Kasir: Tampilkan grand total seluruh belanjaan, lalu panggil kembali metode statis catatLogKasir("Transaksi #042 selesai diproses.");.

Contoh Output:

```
None
[LOG SISTEM KASIR] Membuka transaksi baru untuk Pelanggan #042.
=== DAFTAR TRANSAKSI BIOSKOP ===
- Tiket Reguler (Studio 3) - Interstellar (2 x Rp40000.0) | Total: Rp80000.0
- Tiket VIP Premiere - Dune: Part Two (2 x Rp85000.0) | Total: Rp170000.0
- F&B Snack: Combo Popcorn Caramel + 2 Soda (1 x Rp55000.0) | Total: Rp55000.0
------------------------------------------------------------
TOTAL TRANSAKSI : Rp305000.0
============================================================
[LOG SISTEM KASIR] Transaksi #042 selesai diproses.
```

Jawaban:

ItemTransaksi

```
package Unguided;


public interface ItemTransaksi {
    double getNominalBayar();
    String getNamaTransaksi();


    // Default method: mencetak satu baris struk
    default void cetakRincianItem() {
        System.out.println("- " + getNamaTransaksi() + " | Total: Rp" +
getNominalBayar());
    }


    // Static method: mencatat log kasir
    static void catatLogKasir(String pesan) {
        System.out.println("[LOG SISTEM KASIR] " + pesan);
    }
}
```

Penjelasan:
Interface ItemTransaksi berfungsi sebagai kontrak publik yang harus dipenuhi oleh setiap barang yang dapat diproses oleh kasir. Interface ini mendeklarasikan dua abstract method, yaitu getNominalBayar() untuk mengambil total harga suatu item dan getNamaTransaksi() untuk mengambil deskripsi item tersebut. Selain itu, interface ini memiliki *default method* cetakRincianItem() yang mencetak satu baris struk dengan memanggil kedua method tersebut secara dinamis. Karena bersifat default, method ini otomatis dimiliki oleh seluruh kelas yang mengimplementasikan interface tanpa perlu ditulis ulang di masing-masing kelas. Interface ini juga memiliki *static method* catatLogKasir(String pesan) yang mencetak pesan log sistem kasir dan dipanggil langsung melalui nama interface, tanpa membuat object.

TiketBioskop

```
package Unguided;


public abstract class TiketBioskop implements ItemTransaksi {
    private String judulFilm;
    private double hargaDasar;
    private int jumlahKursi;

    public TiketBioskop(String judulFilm, double hargaDasar, int jumlahKursi) {
        this.judulFilm = judulFilm;
        this.hargaDasar = hargaDasar;
        this.jumlahKursi = jumlahKursi;
    }

    public String getJudulFilm() {
        return judulFilm;
    }
```

```
    public double getHargaDasar() {
        return hargaDasar;
    }

    public int getJumlahKursi() {
        return jumlahKursi;
    }

    public abstract double hitungTotalBayar();

    public abstract String getNamaTransaksi();

    // Kontrak publik kasir, hanya meneruskan hasil hitungTotalBayar()
    @Override
    public double getNominalBayar() {
        return hitungTotalBayar();
    }
}
```

Penjelasan:
Kelas TiketBioskop dideklarasikan sebagai abstract class sehingga tidak dapat diinstansiasi secara langsung, karena tiket film murni belum memiliki jenis studio yang jelas. Kelas ini menyimpan tiga atribut private, yaitu judulFilm, hargaDasar, dan jumlahKursi, yang diinisialisasi melalui constructor menggunakan kata kunci this karena nama parameter sama dengan nama field. Atribut tersebut hanya dapat diakses dari luar kelas melalui getter, sesuai prinsip enkapsulasi. Kelas ini mendeklarasikan dua abstract method, yaitu hitungTotalBayar() dan getNamaTransaksi(), yang perumusannya diserahkan kepada kelas turunan agar tiap jenis tiket memiliki perhitungan dan format struk yang berbeda. Kelas ini juga mengimplementasikan ItemTransaksi dan mewujudkan method getNominalBayar() dengan mengembalikan hasil

hitungTotalBayar(). Dengan cara ini, logika perhitungan internal tiket tetap terpisah dari kontrak publik yang digunakan kasir.

TiketReguler

```
package Unguided;


public class TiketReguler extends TiketBioskop {
    private int nomorStudio;


    public TiketReguler(String judulFilm, double hargaDasar, int jumlahKursi, int
nomorStudio) {
        super(judulFilm, hargaDasar, jumlahKursi);
        this.nomorStudio = nomorStudio;
    }


    @Override
    public double hitungTotalBayar() {
        return getHargaDasar() * getJumlahKursi();
    }


    @Override
    public String getNamaTransaksi() {
        return "Tiket Reguler (Studio " + nomorStudio + ") - " + getJudulFilm()
                + " (" + getJumlahKursi() + " x Rp" + getHargaDasar() + ")";
    }
}
```

Penjelasan:
Kelas TiketReguler merupakan turunan TiketBioskop yang menambahkan atribut nomorStudio. Constructor-nya memanggil super(judulFilm, hargaDasar, jumlahKursi) untuk menginisialisasi atribut milik kelas induk, kemudian mengisi nomorStudio menggunakan this. Method hitungTotalBayar() di-*override* untuk mengembalikan hasil perkalian harga dasar dengan jumlah kursi. Method getNamaTransaksi() di-*override* untuk menghasilkan teks yang memuat nomor studio, judul film, jumlah kursi, dan harga per kursi. Karena atribut milik induk bersifat private, kelas ini mengaksesnya melalui getter seperti getHargaDasar() dan getJudulFilm().

TiketVIP

```
 package Unguided;


public class TiketVIP extends TiketBioskop {
    private double biayaFasilitasVIP;


    public TiketVIP(String judulFilm, double hargaDasar, int jumlahKursi, double
biayaFasilitasVIP) {
        super(judulFilm, hargaDasar, jumlahKursi);
        this.biayaFasilitasVIP = biayaFasilitasVIP;
    }


    @Override
    public double hitungTotalBayar() {
        return (getHargaDasar() + biayaFasilitasVIP) * getJumlahKursi();
    }


    @Override
```

```
    public String getNamaTransaksi() {
        return "Tiket VIP Premiere - " + getJudulFilm() + " (" + getJumlahKursi()
                + " x Rp" + (getHargaDasar() + biayaFasilitasVIP) + ")";
    }
}
```

Penjelasan :
Kelas TiketVIP juga merupakan turunan TiketBioskop dengan atribut tambahan biayaFasilitasVIP, yaitu biaya fasilitas seperti sofa recliner yang dikenakan untuk setiap kursi. Pada method hitungTotalBayar(), harga dasar dijumlahkan terlebih dahulu dengan biaya fasilitas VIP, lalu dikalikan dengan jumlah kursi. Pada method getNamaTransaksi(), harga satuan yang ditampilkan sudah merupakan gabungan harga dasar dan biaya fasilitas, sehingga struk menampilkan harga per kursi yang sebenarnya dibayar penonton.

PaketSnack

```
package Unguided;


public class PaketSnack implements ItemTransaksi {
    private String namaPaket;
    private int jumlahPorsi;
    private double hargaPerPorsi;


    public PaketSnack(String namaPaket, int jumlahPorsi, double hargaPerPorsi) {
        this.namaPaket = namaPaket;
        this.jumlahPorsi = jumlahPorsi;
        this.hargaPerPorsi = hargaPerPorsi;
    }
```

```
    @Override
    public double getNominalBayar() {
        return jumlahPorsi * hargaPerPorsi;
    }

    @Override
    public String getNamaTransaksi() {
        return "F&B Snack: " + namaPaket + " (" + jumlahPorsi + " x Rp" +
hargaPerPorsi + ")";
    }
}
```

Penjelasan :
Kelas PaketSnack merupakan kelas independen yang bukan turunan TiketBioskop, tetapi mengimplementasikan ItemTransaksi. Hal ini menunjukkan bahwa dua entitas dari hierarki yang berbeda dapat diperlakukan sama oleh kasir selama keduanya memenuhi kontrak interface yang sama. Kelas ini memiliki atribut namaPaket, jumlahPorsi, dan hargaPerPorsi yang bersifat private. Method getNominalBayar() mengembalikan hasil perkalian jumlah porsi dengan harga per porsi, sedangkan getNamaTransaksi() mengembalikan teks deskripsi paket beserta kuantitas dan harga satuannya.

Main

```
package Unguided;


public class Main {
    public static void main(String[] args) {
        // 1. Uji proteksi abstraksi (tidak bisa diinstansiasi, error jika dibuka
komentarnya)
        // TiketBioskop tiket = new TiketBioskop("Interstellar", 40000, 2);
```

```
        // 2. Buka kasir
        ItemTransaksi.catatLogKasir("Membuka transaksi baru untuk Pelanggan #042.");

        // 3. Buat objek transaksi
        TiketReguler tiketReguler = new TiketReguler("Interstellar", 40000, 2, 3);
        TiketVIP tiketVIP = new TiketVIP("Dune: Part Two", 60000, 2, 25000);
        PaketSnack paketSnack = new PaketSnack("Combo Popcorn Caramel + 2 Soda", 1, 55000);

        // 4. Array polimorfik
        ItemTransaksi[] keranjang = { tiketReguler, tiketVIP, paketSnack };

        // 5. Pemrosesan dinamis
        System.out.println("=== DAFTAR TRANSAKSI BIOSKOP ===");
        double grandTotal = 0;
        for (ItemTransaksi item : keranjang) {
            item.cetakRincianItem();
            grandTotal += item.getNominalBayar();
        }

        // 6. Tutup kasir
        System.out.println("-----------------------------------------------------------");
        System.out.println("TOTAL TRANSAKSI : Rp" + grandTotal);

System.out.println("==========================================================");
```

```
        ItemTransaksi.catatLogKasir("Transaksi #042 selesai diproses.");
    }
}
```

Penjelasan :

Kelas Main berfungsi sebagai kelas eksekusi untuk menguji seluruh rancangan. Baris instansiasi new TiketBioskop(...) sengaja dijadikan komentar karena akan menyebabkan error kompilasi, sebagai bukti bahwa abstract class tidak dapat dibuat menjadi object. Program kemudian memanggil ItemTransaksi.catatLogKasir() secara langsung melalui nama interface untuk mencatat pembukaan transaksi. Selanjutnya dibuat tiga object, yaitu TiketReguler, TiketVIP, dan PaketSnack, yang disimpan dalam satu array bertipe ItemTransaksi[]. Array ini disebut array polimorfik atau heterogen karena menampung object dari kelas yang berbeda-beda melalui satu tipe referensi yang sama. Pada perulangan for-each, setiap elemen memanggil cetakRincianItem() dan nominal bayarnya dijumlahkan ke variabel grandTotal. Saat method dipanggil, Java secara otomatis menjalankan versi method sesuai tipe object sebenarnya (*dynamic binding*), sehingga format struk tiap item berbeda meski dipanggil dengan cara yang sama. Terakhir, program mencetak total transaksi dan memanggil kembali catatLogKasir() untuk menandai transaksi telah selesai.

Output :

```
run:
[LOG SISTEM KASIR] Membuka transaksi baru untuk Pelanggan #042.
=== DAFTAR TRANSAKSI BIOSKOP ===
- Tiket Reguler (Studio 3) - Interstellar (2 x Rp40000.0) | Total: Rp80000.0
- Tiket VIP Premiere - Dune: Part Two (2 x Rp85000.0) | Total: Rp170000.0
- F&B Snack: Combo Popcorn Caramel + 2 Soda (1 x Rp55000.0) | Total: Rp55000.0
------------------------------------------------------------
TOTAL TRANSAKSI : Rp305000.0
============================================================
[LOG SISTEM KASIR] Transaksi #042 selesai diproses.
BUILD SUCCESSFUL (total time: 0 seconds)
```

Penjelasan :

Output program menampilkan log pembukaan transaksi, daftar tiga item belanjaan, total transaksi, lalu log penutupan transaksi. Item pertama adalah TiketReguler Interstellar dengan total Rp80.000 (40.000 × 2 kursi). Item kedua adalah TiketVIP Dune: Part Two dengan total Rp170.000 ((60.000 + 25.000) × 2 kursi). Item ketiga adalah PaketSnack dengan total Rp55.000 (1 porsi × 55.000). Ketiganya dijumlahkan menjadi grand total Rp305.000. Meskipun berasal dari kelas yang berbeda, semua item dapat diproses dengan cara yang sama lewat array ItemTransaksi[], dan format baris tiap item tetap berbeda karena adanya polimorfisme.

D. Kesimpulan

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa abstract class digunakan sebagai kerangka umum bagi kelas-kelas turunannya. Kelas ini tidak dapat diinstansiasi secara langsung, dan method abstract-nya wajib diimplementasikan oleh subclass. Pada program kasir bioskop, hal ini terlihat pada kelas TiketBioskop yang menyerahkan perumusan hitungTotalBayar() dan getNamaTransaksi() kepada TiketReguler dan TiketVIP, sehingga tiap jenis tiket memiliki perhitungan dan format struk yang berbeda.

Interface berfungsi sebagai kontrak yang dapat dipenuhi oleh kelas-kelas dari hierarki yang berbeda. Interface ItemTransaksi memungkinkan TiketBioskop dan PaketSnack, yang tidak memiliki hubungan pewarisan, diperlakukan sama oleh sistem kasir. Hal ini memungkinkan penerapan polimorfisme melalui array ItemTransaksi[], di mana tiap object menjalankan method sesuai tipe aslinya.

Fitur Java 8 berupa default method dan static method pada interface memungkinkan penambahan fungsi baru, yaitu cetakRincianItem() dan catatLogKasir(), tanpa mengubah kelas-kelas yang sudah mengimplementasikan interface tersebut.

Perbedaan utama kedua konsep ini adalah abstract class dapat memiliki atribut, constructor, dan method berisi implementasi, tetapi hanya mendukung satu pewarisan. Interface hanya mendefinisikan kontrak dan konstanta, tidak memiliki constructor, serta memungkinkan satu kelas mengimplementasikan lebih dari satu interface. Dengan demikian, tujuan praktikum untuk memahami, mengimplementasikan, dan membedakan abstract class dan interface dalam Java telah tercapai.